

Redis Data Types

1. Strings

Description: Strings are the most basic Redis data type, and they are binary-safe, meaning they can contain any type of data (e.g., text, integers, JSON, or binary data like images).

Use Cases:

- Caching: Store small pieces of data like HTML snippets or database query results.
- Counters: Use for incrementing or decrementing values (e.g., page views, votes).
- Simple key-value storage: Store configuration settings or user sessions.

Example Commands:

```
SET user:1:name "Alice"
```

```
GET user:1:name
```

```
INCR page_views
```

```
DECR available_slots
```

```
APPEND user:1:name " Smith"
```

```
STRLEN user:1:name
```

Redis Data Types

2. Lists

Description: Lists are ordered collections of strings, allowing you to add elements to the head (left) or tail (right) of the list.

Use Cases:

- Message Queues: Implement a simple message queue by pushing to the list and popping from it.
- Recent Items: Keep track of recent actions like the last 100 logged events.
- Task Lists: Store tasks and process them in order.

Example Commands:

LPUSH tasks "task1"

RPUSH tasks "task2"

LPOP tasks -- Removes and returns the first element (task1)

RPOP tasks -- Removes and returns the last element (task2)

LRANGE tasks 0 -1 -- Returns all elements in the list

LLEN tasks -- Returns the length of the list

Redis Data Types

3. Sets

Description: Sets are collections of unique, unordered strings. They are useful when you need to store unique items.

Use Cases:

- Tags: Store unique tags or categories.
- Membership: Quickly check if an item is a member of a set.
- Common Elements: Find common elements between multiple sets (e.g., mutual friends).

Example Commands:

```
SADD myset "element1"
```

```
SADD myset "element2"
```

```
SADD myset "element1" -- element1 won't be added again
```

```
SISMEMBER myset "element1"
```

```
SMEMBERS myset -- Returns all members of the set
```

```
SUNION set1 set2 -- Returns the union of set1 and set2
```

```
SINTER set1 set2 -- Returns the intersection of set1 and set2
```

```
SCARD myset -- Returns the number of elements in the set
```

Redis Data Types

4. Sorted Sets (Zsets)

Description: Sorted sets are similar to sets but each element is associated with a score, and elements are ordered by their scores.

Use Cases:

- Leaderboards: Implement a ranking system.
- Task Scheduling: Schedule tasks with priorities.
- Rate Limiting: Track actions and their timestamps.

Example Commands:

ZADD leaderboard 100 "Alice"

ZADD leaderboard 200 "Bob"

ZRANGE leaderboard 0 -1 WITHSCORES -- Returns all elements in the sorted set with their scores

ZREM leaderboard "Alice" -- Removes Alice from the leaderboard

ZREVRANGE leaderboard 0 -1 WITHSCORES -- Returns all elements in reverse order

ZSCORE leaderboard "Bob" -- Returns the score of Bob

ZCARD leaderboard -- Returns the number of elements in the sorted set

ZRANGEBYSCORE leaderboard 100 200 -- Returns elements with scores between 100 and 200

Redis Data Types

5. Hashes

Description: Hashes are maps of fields and values, which are useful for representing objects.

Use Cases:

- User Profiles: Store user attributes like name, age, and email.
- Configuration: Store application configuration settings.
- Data Modeling: Represent objects with multiple attributes.

Example Commands:

```
HSET user:1 name "Alice"
```

```
HSET user:1 age 30
```

```
HGET user:1 name
```

```
HGETALL user:1      -- Returns all fields and values in the hash
```

```
HDEL user:1 age      -- Deletes the age field from the hash
```

```
HMSET user:1 name "Bob" age 25 -- Sets multiple fields at once
```

```
HLEN user:1          -- Returns the number of fields in the hash
```

```
HINCRBY user:1 age 1 -- Increments the age field by 1
```